RUNI – Algorithms – Homework 4

Due date: Sunday 07/12/2025

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Problem 1 (20 pts):

Let $G = (V, E)$ be a directed graph, $w: E \to \mathbb{R}$ a weight function on the edges such that no negative cycle exists in G. Assume that $d(u, v)$ is known for all $u, v \in V$, and provided in a matrix form so that $d(u, v)$ can be accessed in $O(1)$ by reading $d[u][v]$. Let $e = (a, b)$ with given weight $w(a, b)$ be an edge that does **not** exist in E, and let $G' = (V, E \cup e)$.

a. Describe an algorithm that computes the shortest path distances $d(u, v)$ in graph G' for all $u, v \in V$ in $O(|V|^2)$ time.

Algorithm:

For all $u \in V$ do:

For all $v \in V$ do:

$$d(u, v) = \min\{d[u][v], d[u][a] + w(a, b) + d[a][v]\}$$

b. Prove the correctness of your algorithm.

Correctness:

Let P be an acyclic shortest path from u to v in G'. We will consider two options:

1) P does not include the edge (a, b).

In that case, all the edges of P appear in G as well and thus P is a path in G. We will show that P is a shortest path in G.

Assume towards contradiction that P is not a shortest path in G, and let P' be a shortest u->v path in G. Since all the edges of G are in G', P' exists in G' and that is a contradiction to P being a shortest path in G'. Knowing that P is a shortest path both in G and G' we conclude that in this case, $d(u, v) = d[u][v]$.

2) P does include the edge (a, b).

We will rewrite P as the concatenation $P_1 \circ P_2$ where $P_1 = (u, \dots, a)$ and $P_2 = (b, \dots, v)$. Notice that since P_1 and P_2 are sub-paths of a shortest path, then they are shortest paths themselves, So $w(P_1) = d(u, a)$ and $w(P_2) = d(b, v)$.

By a similar proof to that of case 1) we have that $d(u, a) = d[u][a]$ and $d(b, v) = d[b][v]$.

Concluding case 2) we get

$$d(u, v) = w(P) = w(P_1) + w(a, b) + w(P_2)$$

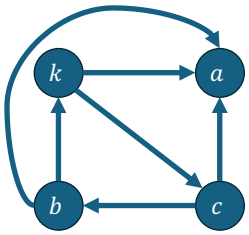
$$= d(u, a) + w(a, b) + d(b, v) = d[u][a] + w(a, b) + d[b, v].$$

Since every path in G' either includes or does not include (a, b), the shortest path from u to v in G' is the minimum of the shortest path including (a, b) and the shortest path not including (a, b), which is equal to

$$\min\{d[u][v], d[u][a] + w(a, b) + d[b][v]\}.$$

Problem 2 (20 pts):

Recall that a **tournament graph** is a simple directed graph where the underlying undirected graph is a clique. Let $G = (V, E)$ be a tournament graph. A vertex $k \in V$ is called a ***king*** if for every other vertex $v \in V$ there is a directed path of length at most 2 from k to v. In the diagram there is an example tournament graph over 4 vertices, with 3 kings (k, b, c).



a. Claim True/False and shortly Justify: The DFS run starting at a king vertex will create a single tree.

True. Any king is a root vertex of G.

b. Claim True/False and shortly Justify: Any BFS run of a tournament graph G that has a king vertex will yield a forest with a single tree.

False. Run BFS starting on vertex (a) in the example.

Only BFS runs starting at a root vertex will create a spanning tree.

c. Provide a simple algorithm that given a vertex $u \in V$ of G runs in time $O(m)$, where $m = |E|$, and correctly decides if u is a king of G or not. shortly justify correctness and analyze runtime.

As this is a tournament graph, we know that $m = O(n^2)$, therefore $n = O(m)$.

Run BFS starting at u and check for all vertices $v \in V$, $d(u, v) \leq 2$. If so, u is a king, otherwise it is not.

The correctness follows from the shortest paths correctness of BFS and the definition of a king.

The runtime is $O(m + n) = O(m)$.

It has been shown that every tournament graph is guaranteed to have at least one king vertex.

d. Provide a simple algorithm that runs in time $O(n^3)$ and finds a king in the graph G.

Repeat the algorithm of item (a) for every vertex until a king is found.

Runtime is $O(m \cdot n) = O(n^3)$.

Correctness follows from the correctness of (a).

Let $x \in V$ be some vertex. We say that x *beat* another vertex $y \in V$ if the edge (x, y) exists in the tournament graph's edges E. Let $L_x \subseteq V$ be the set of vertices that x beats, and $W_x \subseteq V$ be the set of vertices that beat x. Note that the out-degree of x is exactly $|L_x|$.

e. Complete the proof of the following claim:

if x has the maximum out-degree of all vertices, then x is a king.

All vertices in L_x are distance 1 from x by definition. We show that all vertices in W_x are distance 2 from x.

Let vertex $u \in W_x$. Assume towards contradiction that there does not exist any $w \in L_x$ with $(w, u) \in E$.

Thus there is not a directed edge from w to u. Since this is a tournament graph, this means that there is a directed edge from u to w. This is true for all $w \in L_x$ so we have that $L_x \subset L_u$. As this is a tournament graph we know that $x \in L_u$.

But this means that $|L_u| > |L_x|$

Which contradicts x having the maximum out-degree.

f. Provide an algorithm that runs in time $O(n + m)$ and finds a king in the graph G. State the correctness claim for this algorithm (no need to prove) and shortly analyze its runtime.

Algorithm:

1. Count the out-degree of every vertex by iterating over the graph (edges and vertices). This takes O(m+n)

2. Find the maximum out-degree.

3. Return the first vertex with the maximum out-degree

Steps 2-3 can be done in O(n) by finding the maximum element in an unsorted array.

Correctness claim:

The algorithm finds a maximum out-degree vertex. We know by item (c) that this vertex is a king vertex.

Problem 3 (20 pts):

Let $G = (V, E)$ be a **directed acyclic graph** (DAG). Let $s \in V$ and let $w: E \to \mathbb{R}^+$ be a non-negative weight function. Describe an efficient algorithm to find every for vertex $v \in V$, a path from s with **maximum** weight.

Justify the correctness of your algorithm and analyze its runtime.

Algorithm:

1. Multiply all weights in G by -1.
2. Find the shortest path from s to all vertices $v \in V$ using the DAG shortest path algorithm from class.

Correctness:

Since G is acyclic there are no negative weight cycles after step 1, and it remains a DAG after this step.

Every shortest path given by the algorithm corresponds to the maximum distance path.

The correctness of the paths being shortest follows from the correctness of the algorithm from class.

Runtime:

Changing the weights takes $O(m + n)$, since this requires reading the graph.

The algorithm from class requires $O(m + n)$.

In total $O(m + n)$.

Problem 4 (20 pts):

Let $G = (V, E)$ be a directed graph with real weights $w: \boldsymbol{V} \cup \boldsymbol{E} \rightarrow \mathbb{R}$ on both its **edges and its vertices**. The weight of a path $P = s, v_1, \ldots, v_n, t$ is the sum of the weights of its edges **and the weights of its vertices**. Note that the weights of s and t are also included.

Given $G = (V, E), w$ and $s \in V$, suggest an algorithm that computes the shortest path from s to any other vertex $v \in V$. Justify the correctness of your algorithm and analyze its runtime.

Algorithm:

1. Construct $G' = (V', E')$ with weight function w' on the edges as follows.
   - $V' = V_{in} \cup V_{out}$, where each is a copy of V: $V_{in} = \{v_{in} | v \in V\}, V_{out} = \{v_{out} | v \in V\}$.
   - $E' = \{(u_{out}, v_{in}) | (u, v) \in E\} \cup \{(v_{in}, v_{out}) | v \in V\}$
   - $w' = \begin{cases} w(v) \; if \; e = (v_{in}, v_{out}) \\ w(u, v) \; if \; e = (u_{out}, v_{in}) \end{cases}$
2. Run BF on G' with s_{in} as the source vertex.
3. Return the values obtained for vertices v_{out}.

Correctness:

Let d be some real value. We prove that a path of weight d exists from s_{in} to t_{out} in G' iff some path of weight d exists from s to t in G.

First direction: Suppose that $P = (s, v^1, \ldots, v^n, t)$ is the shortest path from s to t in G. Then we have a similar path with the same weight in G': $P = (s_{in}, v^1_{out}, \ldots, v^n_{in}, t_{out})$, since the edges between the vertices remain the same and the vertex weights are replaced by the weights of the (v_{in}, v_{out}) edges.

The second direction is similar.

Thus, by the shortest path correctness of BF in G', we have the shortest paths in G.

Runtime:

Construction of the graph G' requires $O(|E'| + |V'|) = O(m' + n') = O(m + n)$.

The BF run requires $O(|E'| \cdot |V'|) = O(m' \cdot n') = O(m \cdot n)$.

In total $O(m \cdot n)$.

The following problems assume knowledge of the Floyd-Warshall algorithm to be taught in week 6.

Problem 5 (20 pts):
Run the Floyd-Warshall algorithm on the graph given below by δ_0. Describe (fill in) δ_i for each step.

δ_0	1	2	3	4	5
1	0	5	1	∞	∞
2	∞	0	∞	-2	∞
3	2	∞	0	∞	∞
4	8	∞	4	0	∞
5	∞	∞	4	2	0

δ_1	1	2	3	4	5
1					
2					
3					
4					
5					

δ_2	1	2	3	4	5
1					
2					
3					
4					
5					

δ_3	1	2	3	4	5
1					
2					
3					
4					
5					

δ_4	1	2	3	4	5
1					
2					
3					
4					
5					

δ_5	1	2	3	4	5
1					
2					
3					
4					
5					

(Answer for problem 5):

δ_0	1	2	3	4	5
1	0	5	1	∞	∞
2	∞	0	∞	-2	∞
3	2	∞	0	∞	∞
4	8	∞	4	0	∞
5	∞	∞	4	2	0

δ_1	1	2	3	4	5
1	0	5	1	∞	∞
2	∞	0	∞	-2	∞
3	2	7	0	∞	∞
4	8	13	4	0	∞
5	∞	∞	4	2	0

δ_2	1	2	3	4	5
1	0	5	1	3	∞
2	∞	0	∞	-2	∞
3	2	7	0	5	∞
4	8	13	4	0	∞
5	∞	∞	4	2	0

δ_3	1	2	3	4	5
1	0	5	1	3	∞
2	∞	0	∞	-2	∞
3	2	7	0	5	∞
4	6	11	4	0	∞
5	6	11	4	2	0

δ_4	1	2	3	4	5
1	0	5	1	3	∞
2	4	0	2	-2	∞
3	2	7	0	5	∞
4	6	11	4	0	∞
5	6	11	4	2	0

δ_5	1	2	3	4	5
1	0	5	1	3	∞
2	4	0	2	-2	∞
3	2	7	0	5	∞
4	6	11	4	0	∞
5	6	11	4	2	0